

Insurance Invoice Auditing

Assessment write-up: measurement, uncertainty, and next steps

Approach and measurement

I built the solution as a two-stage pipeline. First, an LLM reads each hospital's Markdown contract and converts it into one consistent schema containing services, rates, units, effective dates, premiums, discounts, bundles, exclusions, quantity limits, amendments, calculation order, and source evidence. Python validates the extraction before it can be used. The checks cover schema correctness, quoted evidence, source coverage, service references, numeric values, date ranges, amendments, and rule ordering. An invalid extraction is retried for the affected chunk and otherwise stops visibly; it is never accepted silently.

Second, the invoice engine maps each free-text line description to a canonical service, selects the effective rules, and calculates the expected amount using integer cents and half-up rounding. Python also performs invoice-level and historical checks for arithmetic errors, duplicate invoices and lines, bundles, exclusion windows, daily caps, and cumulative thresholds. The LLM therefore handles language interpretation, while validation, arithmetic, and enforceable business rules remain deterministic and testable.

Hospital 1 was the labelled development set. I compared each generated prediction with the supplied label and measured invoice-level classification accuracy, precision, recall, and F1. I also measured whether the predicted expected total exactly matched the labelled expected total and reviewed results by error category and systematic failure type.

Hospital 1 measure	Result
Labelled invoices evaluated	913
Classification accuracy	100.0%
Precision / recall / F1	100.0% / 100.0% / 100.0%
Exact expected-total accuracy	97.8%

The classification result shows that the pipeline identified every labelled erroneous invoice without false positives on this development set. The lower expected-total result is important: correctly deciding that an invoice is wrong does not guarantee that every component of the corrected amount is exact. Category-level review also showed overlap between generic price mismatches and more specific premium, discount, or unit errors.

These figures are development results, not an unbiased estimate of performance on a new hospital, because Hospital 1 was used while refining the implementation. Hospitals 2 and 3 have no labels, so I report coverage rather than unsupported accuracy claims: the final files contain predictions for all 1,125 Hospital 2 invoices and all 932 Hospital 3 invoices. I intentionally did not submit Hospitals 4 and 5 rather than provide thinly validated coverage.

Uncertainty and limitations

The highest-risk step is contract extraction: one confidently wrong rate could affect many invoices. Every extracted financial or enforceable fact must therefore include source evidence, and a contract is cached only after passing deterministic validation. A cache is reused only when the contract content, provider, model, prompt, and schema versions match.

Service matching is the second major uncertainty. Hospital descriptions are abbreviated and may omit the specialty or care setting. The matcher records its best candidate, runner-up, score, and margin. Weak or close matches lower invoice confidence or abstain instead of being presented as certain. Price has only weak influence on matching because the billed price may itself be the error being audited.

The main remaining limitations are:

- A description that omits its distinguishing clinical meaning may not be recoverable by deterministic matching.
- One monetary discrepancy can reasonably receive both a generic mismatch category and a specific rule category.
- Malformed dates prevent reliable selection of dated amendments or weekend rules; these cases are flagged rather than assigned an invented rate.
- Duplicate allocation and cumulative discounts depend on a stable chronological ordering when the contract does not completely specify tie-breaking.
- Hospital 1's perfect classification should not be treated as proof of generalisation to differently written contracts.

What I would do with another week

First, I would run the same extraction evaluation across several models, including a smaller and cheaper model. I would add a deterministic availability fallback that produces the same canonical schema and must pass exactly the same evidence, coverage, reference, date, numeric, and amendment checks. Its provenance and lower confidence would be explicit; it would never silently replace a failed LLM extraction.

Second, I would create a small adjudicated set of ambiguous line descriptions and evaluate service matching independently using top-1 accuracy, abstention rate, and confidence calibration. An LLM resolver could then be restricted to only the low-margin queue instead of being used for all invoice lines.

Third, I would reserve part of Hospital 1 as a locked holdout or create contract perturbation tests so prompt and rule-engine changes are not measured on the same examples used for development. I would also test whether confidence values correspond to observed correctness, not only whether classifications are correct.

Finally, I would extend the same evidence-first implementation to Hospitals 4 and 5, add more contract-shape regression tests, and refine category prioritisation so the main diagnosis remains concise while detailed contributing findings stay available in the audit trace.